

# **Project Report**

Course:

**Fundamentals In AI ML**

(Course Code – CSA2001)

Faculty – Dr. Satyabrata Nath

MADE BY :-

**SHOUBHIT PRAMANICK**

Reg No.: 25BCE10083

Branch: CSE Core

School – SCOPE

Date of Submission – 31/03/2026

# 1. Introduction

This project presents a Rock Paper Scissors game implemented in Prolog. Unlike conventional imperative languages, Prolog is a logic programming language based on facts and rules, offering a different approach to problem-solving. The objective was to develop a functional, interactive game that demonstrates key concepts such as recursion, user input handling, random selection, and pattern matching. The game allows a user to play against the computer in a command-line interface, with the option to continue multiple rounds until quitting. This small-scale project serves as a practical exercise in logic programming while reinforcing principles of digital literacy and version control through GitHub.

This project is my little experiment: a command-line game where you play against the computer. You type rock., paper., or scissors., the computer picks randomly, and the program tells you who won. You can keep playing until you type quit.. It's simple, but building it taught me about recursion, user input, randomness, and how to write a clean README. Most importantly, it showed me that learning a new programming paradigm can be both fun and rewarding – and that even a tiny project deserves to be shared properly on GitHub.

---

## 2. Project Overview

### Goal:

Build an interactive Rock Paper Scissors game in Prolog that runs in the terminal, accepts user input, generates a random computer move, determines the winner, and loops until the user quits.

### Tools Used:

- **SWI-Prolog** – the programming language and environment.
- **Git & GitHub** – to track changes and publish my project online.
- **Markdown** – to write documentation (README, project statement).

## Deliverables:

- rock\_paper\_scissors.pl – the Prolog source code.
  - README.md – an explanation of how to run the game.
  - project\_statement.md – why I built it and what I learned.
  - requirements.txt – a note that only SWI-Prolog is needed.
  - This report.
- 

## 3. Code Explanation

### 3.1 The Game Loop

The game starts with main/0, which prints a welcome message and calls play/0:

```
prolog
```

```
main :-
```

```
    write('Rock-Paper-Scissors Game'), nl,
    write('Enter your choice (rock/paper/scissors) or
"quit" to stop. '), nl,
    play.
```

play/0 reads the user's input and decides what to do next:

```
prolog
```

```
play :-
```

```
    write('Your move: '),
    read(User),
    (   User == quit
->    write('Thanks for playing!'), nl
```

```

;   valid_move(User)
->  random_move(Comp),
    write('Computer chose: '), write(Comp), nl,
    result(User, Comp, Outcome),
    write(Outcome), nl,
    play
;   write('Invalid move. Try again. '), nl,
    play
).
```

- If the user types quit., the game ends.
- If the move is valid, the computer picks a random move, and the result is shown. Then play calls itself – that's how the loop continues.
- If the move is invalid, it prints an error and loops again.

## **3.2 Valid Moves**

I defined a simple fact to recognise valid moves:

```

prolog
valid_move(rock).
valid_move(paper).
valid_move(scissors).
```

## **3.3 Random Computer Move**

SWI-Prolog has a built-in random/3 that gives a random integer. I map 0,1,2 to rock, paper, scissors:

```

prolog
```

```
random_move(Move) :-  
    random(0, 3, N),  
    nth0(N, [rock, paper, scissors], Move).
```

### **3.4 Determining the Winner**

Instead of writing a bunch of if-else statements, I used Prolog's pattern matching to cover all nine possibilities. Each rule says: if the user picks X and the computer pick Y, then this is the outcome.

```
prolog  
result(rock, scissors, 'You win! Rock crushes  
scissors.').  
result(paper, rock, 'You win! Paper covers rock.').  
result(scissors, paper, 'You win! Scissors cut  
paper.').  
result(rock, paper, 'Computer wins! Paper covers  
rock.').  
% ... and so on
```

This made the code very easy to read and extend (if I ever wanted to add Lizard and Spock).

---

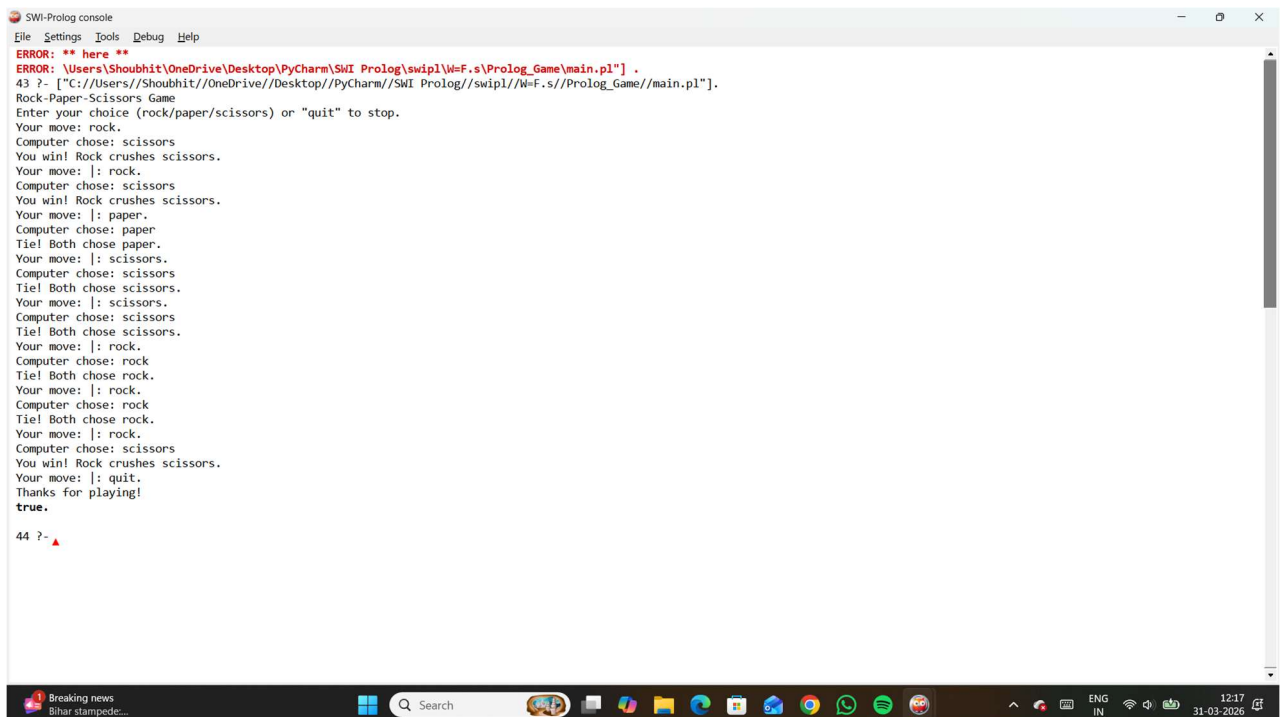
## 4. How to Run the Program

I made sure that anyone with SWI-Prolog installed can run it in a few steps:

1. **Download or clone** the repository from GitHub.
2. **Open a terminal** (or command prompt) and go to the project folder.
3. **Start SWI-Prolog** by typing `swipl`.
4. **Load the file:** `[rock_paper_scissors]`.
5. **Start the game:** `main`.

That's it! No extra libraries, no complicated setup. I even included a `requirements.txt` file that simply says "SWI-Prolog" and how to install it.

## Results



```
SWI-Prolog console
File Settings Tools Debug Help
ERROR: ** here **
ERROR: \Users\Shoubhit\OneDrive\Desktop\PyCharm\SWI Prolog\swipl\W=F.s\Prolog_Game\main.pl" .
43 ?- ["C:/Users/Shoubhit/OneDrive/Desktop/PyCharm/SWI Prolog/swipl/W=F.s/Prolog_Game/main.pl"].
Rock-Paper-Scissors Game
Enter your choice (rock/paper/scissors) or "quit" to stop.
Your move: rock.
Computer chose: scissors
You win! Rock crushes scissors.
Your move: |: rock.
Computer chose: scissors
You win! Rock crushes scissors.
Your move: |: paper.
Computer chose: paper
Tie! Both chose paper.
Your move: |: scissors.
Computer chose: scissors
Tie! Both chose scissors.
Your move: |: scissors.
Computer chose: scissors
Tie! Both chose scissors.
Your move: |: rock.
Computer chose: rock
Tie! Both chose rock.
Your move: |: rock.
Computer chose: rock
Tie! Both chose rock.
Your move: |: rock.
Computer chose: scissors
You win! Rock crushes scissors.
Your move: |: quit.
Thanks for playing!
true.
44 ?- ▲
```

## 5. Testing and Examples

I tested the game thoroughly by playing many rounds and trying edge cases like typing nonsense, entering quit., and making sure the computer's random move worked. Here's a sample session:

```
text
```

```
?- main.
```

```
Rock-Paper-Scissors Game
```

```
Enter your choice (rock/paper/scissors) or "quit" to  
stop.
```

```
Your move: rock.
```

```
Computer chose: paper
```

```
Computer wins! Paper covers rock.
```

```
Your move: scissors.
```

```
Computer chose: scissors
```

```
Tie! Both chose scissors.
```

```
Your move: quit.
```

```
Thanks for playing!
```

```
true.
```

---

## 6. Reflection and Challenges

### What I Learned

- **Prolog is different.** I was used to imperative languages like Python or C. Writing in Prolog forced me to think in terms of facts, rules, and recursion. It felt strange at first, but once I got the hang of it, it was very satisfying.
- **Handling user input in Prolog** was simpler than I thought – `read/1` reads any Prolog term, so typing `rock.` works perfectly.
- **Randomness** was easy with `random/3`, but I had to remember that Prolog's random is deterministic unless you seed it. For this game, it's fine.
- **Version control matters.** Using Git to track changes and GitHub to share my work made me feel like a real developer. Writing a good README also helped me clarify my own thoughts.

### Challenges

The biggest challenge was debugging the recursion in `play/0`. At first, I accidentally made it stop after one round because I forgot to call `play` again. I also had to be careful about using the period in user input – Prolog requires it, so I made sure to mention that in the instructions.

---

## 7. Future Improvements

If I had more time, I would add:

- **Score tracking** – keep count of wins, losses, and ties.
- **Best-of-three mode** – play a set number of rounds automatically.
- **Better random seeding** – so the computer's moves are less predictable.
- **A graphical interface** – but that's a bigger project for another day.

Even without these, the game is complete and fun to play.

---



## 8. Conclusion

This project successfully implemented a Rock Paper Scissors game in Prolog, demonstrating key concepts of declarative programming, recursion, and user input handling. The use of Prolog's pattern matching simplified the win-determination logic. Challenges included ensuring the recursive game loop functioned correctly and managing input format expectations. The project was documented and shared via GitHub, reinforcing version control and documentation practices. Future enhancements could include score tracking and multi-round modes. Overall, the project achieved its objectives while expanding proficiency in logic programming and project presentation.

## 9. References

1. **SWI-Prolog.** (n.d.). *SWI-Prolog official website*. Retrieved March 31, 2026, from <https://www.swi-prolog.org>
2. **SWI-Prolog Documentation.** (n.d.). *Library random*. Retrieved March 31, 2026, from